

ملخص مبسط شامل

هياكل البيانات

DATA STRUCTURES

من محاضرات المستودع: 1 · 2 · 4 · 5 و 6
تعريفات · تشبيهات · جداول تعقيد · رسوم توضيحية أصلية

الفصل الأول — المقدمة ولغة الكفاءة

ما هي هياكل البيانات؟ هي طريقة متخصصة لتنظيم البيانات ومعالجتها واسترجاعها وتخزينها بكفاءة. ثلاث كلمات تلخصها بالكامل: نظم (ترتيب منطقي للبيانات)، خزن (في الذاكرة)، وبكفاءة (سرعة أعلى وذاكرة أقل). المشكلة بدون هيكل: بحث بطيء لأنك قد تفحص كل عنصر واحدا تلو الآخر، تحديث صعب لأن الإدراج والحذف يتحولان إلى فوضى، وهدر في الذاكرة بسبب تخصيص غير ذكي للمساحة.

ثلاثة هياكل أساسية بتشبيهات يومية

- المكدس Stack: كومة أطباق — تضيف الطبق من الأعلى وتأخذه من الأعلى. آخر داخل أول خارج (LIFO).
- الطابور Queue: طابور البنك — أول شخص يصل أول شخص يخدم. أول داخل أول خارج (FIFO).
- المصفوفة Array: صف خزائن مرقمة — لكل عنصر مكان ثابت ومعلوم لا يتغير.

الفكرة الجوهرية

نفس البيانات مع هيكل مختلف تعني أداء مختلفا تماما. قائمة أسماء غير مرتبة تعني بحثا بطيئا، بينما القائمة المرتبة تتيح بحثا سريعا. الهيكل ليس شكلا جماليا — إنه قرار أداء.

النوع المجرد للبيانات ADT

ال ADT هو الفكرة: ماذا يفعل هذا النوع؟ أما هيكل البيانات فهو التنفيذ: كيف نبني تلك الفكرة في الذاكرة؟ التشبيه: أزرار الريموت كنترول — أنت تعرف وظيفة كل زر دون أن تحتاج لمعرفة الدائرة الكهربائية داخل الجهاز.

التعقيد والأداء

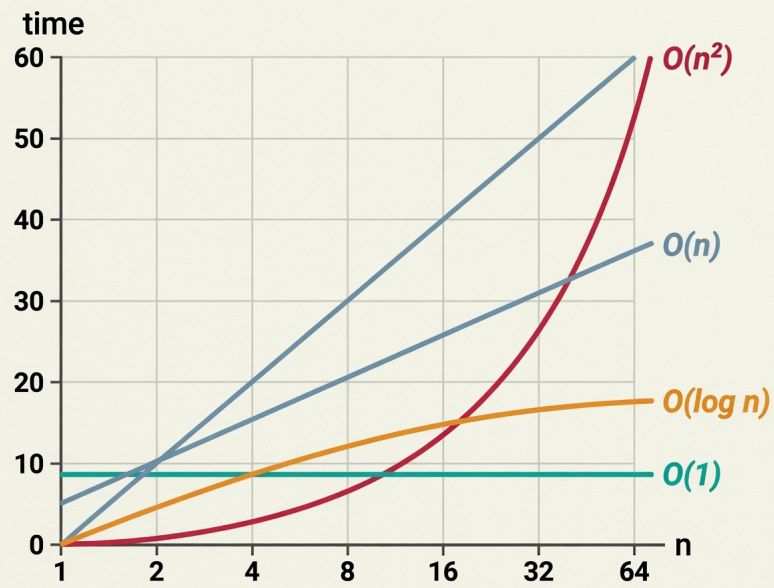
قيدان يحكمان كل خوارزمية: التعقيد الزمني (كم عملية تنفذ مع نمو البيانات؟) والتعقيد المكاني (كم ذاكرة إضافية تحتاج؟). وفي الواقع العملي غالبا نقايض أحدهما بالآخر: ذاكرة أكثر مقابل سرعة أعلى.

ترميز Big-O

ترميز رياضي يصف أداء الخوارزمية في أسوأ حالة. لماذا الأسوأ تحديدا؟ لأن معرفة الحد الأسوأ تضمن لك موثوقية البرنامج مهما كان شكل المدخلات.

مثال	المعنى	الترميز
الوصول لفهرس مصفوفة	ثابت — نفس الوقت مهما كبر الحجم	$O(1)$
البحث الثنائي	لوغاريتمي — تتخلص من نصف المشكلة كل خطوة	$O(\log n)$
البحث عن عنصرين	خطي — الوقت يكبر مع كبر البيانات	$O(n)$
فرز الفقاعات	تربيعي — يتضاعف بشدة مع النمو	$O(n^2)$

Algorithm Time Complexity Growth Rates



منحنيات النمو: عند مليون عنصر، الخوارزمية الخطية تنفذ مليون عملية بينما اللوغاريتمية نحو 20 عملية فقط — هذا هو جوهر المادة كلها.

الفصل الثاني – الهياكل الختية

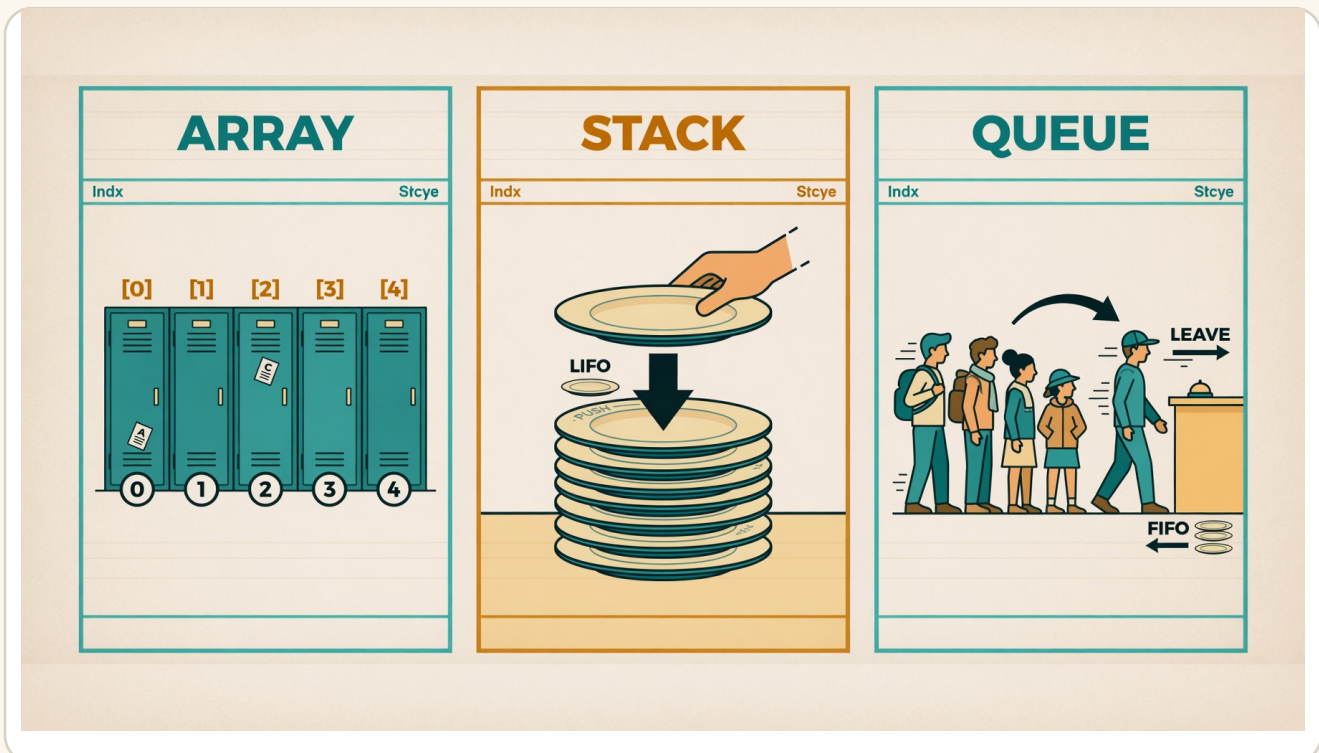
المصفوفة Array

مجموعة عناصر مخزنة بترتيب ومتاحة عبر الفهرس. في المصفوفة [40, 30, 20, 10] القيمة 10 تقع في الفهرس 0 والقيمة 30 في الفهرس 2. يمكن أن تكون أحادية البعد (قائمة بسيطة) أو ثنائية البعد (جدول كمصفوفة رياضية).

عمليات المصفوفة وتعقيدها

التعقيد	كيف تتم؟	العملية
$O(1)$	مباشرة عبر الفهرس	الوصول Access
$O(n)$	فحص عنصر عنصر	البحث Search
$O(n)$	إزاحة العناصر لفتح مكان	الإدراج Insert
$O(n)$	إزاحة العناصر لسد الفراغ	الحذف Delete

لماذا الوصول $O(1)$ ؟ لأن عناصر المصفوفة كتلة ذاكرة متصلة كعربات قطار متلاصقة، فيحسب الحاسوب موقع أي فهرس بمعادلة مباشرة دون المرور على العناصر السابقة. لكن القيود واضحة: حجم ثابت، والإدراج والحذف يتطلبان إزاحة – لذلك ليست مرنة.



الخزائن للمصفوفة، الأطباق للمكدس، وطابور البنك للطابور – ثلاثة تشبيهات تختصر الهياكل الختية.

المكدس Stack — آخر داخل أول خارج

هيكل خطي تتبع عملياته ترتيب LIFO. العمليات الثلاث: Push لإضافة عنصر للقمة، Pop لإزالة عنصر القمة، Peek لمشاهدة القمة دون إزالتها.

```
C#
Stack<int> stack = new Stack<int>();
stack.Push(10);
stack.Push(20);
Console.WriteLine(stack.Peek()); // 20
stack.Pop();
```

الطابور Queue — أول داخل أول خارج

كطابور انتظار حقيقي: Enqueue تضيف للعنصر في مؤخرة الطابور، Dequeue تزيل العنصر من المقدمة، و Front أو Peek كلاهما $O(1)$. تعرض أول عنصر. أداؤها ممتاز Peek.

- أنواع الطابور: بسيط، دائري (ذكر مرورا)، طابور أولويات (ذكر مرورا).
- التطبيقات: أنظمة الجدولة، أنظمة التشغيل، وخوارزمية البحث بالعرض BFS.

الفصل الثالث — العودية Recursion

تقنية برمجية تستدعي فيها الدالة نفسها، وتقسم فيها المشكلة إلى نسخ أصغر من نفس المشكلة حتى تصبح بسيطة الحل مباشرة.

- تشبيهات من الواقع: مرآة تقابل مرآة، الدمى الروسية المتداخلة، مجلد داخل مجلد، وأغصان شجرة تتفرع لأغصان أصغر.
- لماذا نستخدمها؟ تبسط المشاكل المعقدة، وتجعل الكود أقصر، وهي الحل الطبيعي للهياكل الهرمية كالشجرات.

ركنا العودية

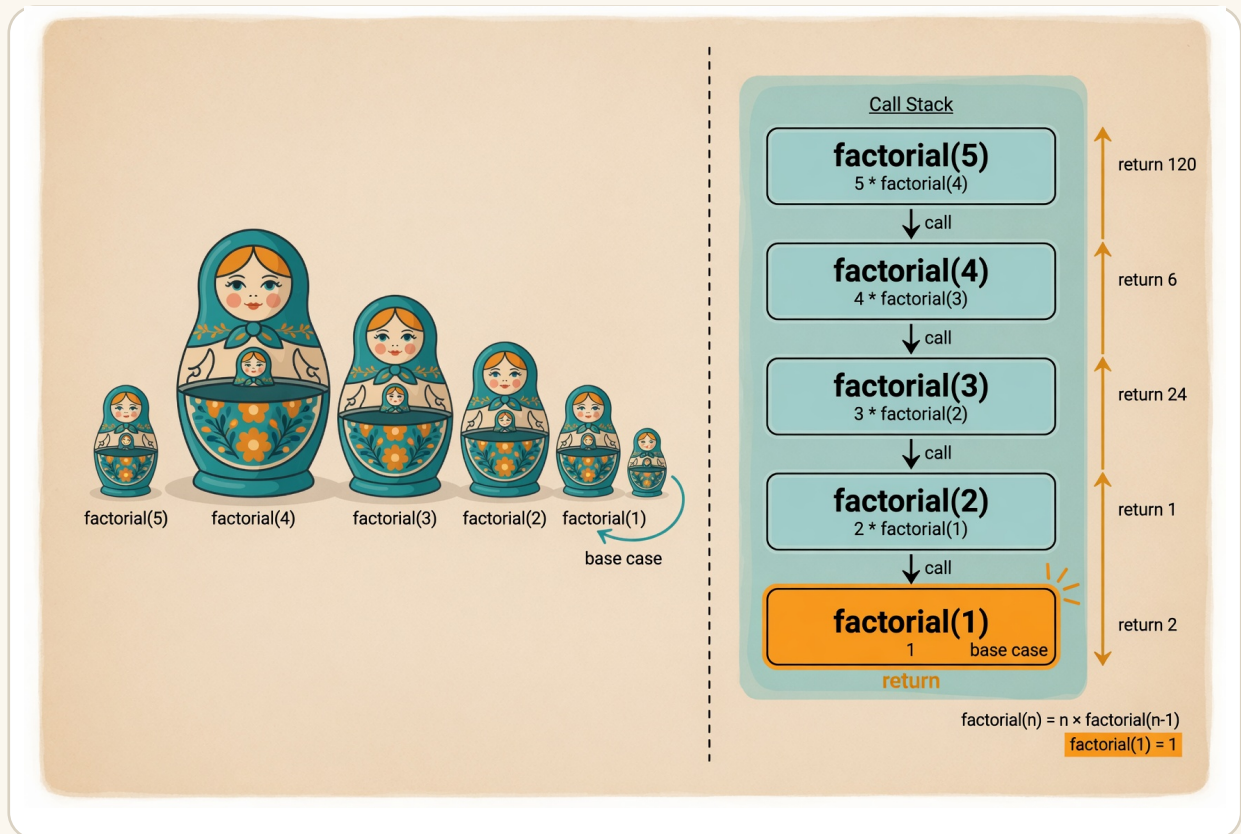
- حالة الأساس Base Case: شرط التوقف — اللحظة التي تصبح فيها المشكلة بسيطة بما يكفي لحلها مباشرة.
 - الحالة العودية Recursive Case: الدالة تعمل على نسخة أصغر من المشكلة وتقترب خطوة من حالة الأساس.
- مثال المضروب: $5!$ يستدعي $4!$ ثم $3!$ ثم $2!$ ثم $1!$ ، وعند حالة الأساس يبدأ الإرجاع بالتصاعد حتى تكتمل النتيجة.

الخطر: العودية اللانهائية

بلا حالة أساس تستمر الدالة في استدعاء نفسها، فتزداد الذاكرة المستهلكة مع كل استدعاء حتى ينهار البرنامج بانهايار المكس Stack Overflow.

كيف تعمل داخليا؟

كل استدعاء يخزن فوق سابقه في مكس الاستدعاءات، وبعد بلوغ حالة الأساس يعود التنفيذ عكسيا من الأقدم للأحدث، وكل طبقة ترجع نتيجتها للطبقة التي تحتها.



الدمى الروسية تشرح الفكرة، ومكدس الاستدعاءات يشرح الآلية: نزول بالاستدعاءات ثم صعود بالإرجاع.

العودة مقابل التكرار Loops

المعيار	العودة	التكرار
الأنسب لـ	الشجرات والمشاكل الهرمية	المهام الخطية البسيطة
طول الكود	أقصر وأوضح	أطول غالبا
الذاكرة	تستهلك مكدس استدعاءات	أوفر

القواعد الذهبية الثلاث

- يجب أن توجد حالة أساس دائما.
- يجب تقليص حجم المشكلة في كل استدعاء.
- يجب أن يتحرك كل استدعاء نحو شرط التوقف.

والا فالنتيجة

عودة لانتهائية ثم انهيار المكدس Stack Overflow ثم فشل البرنامج.

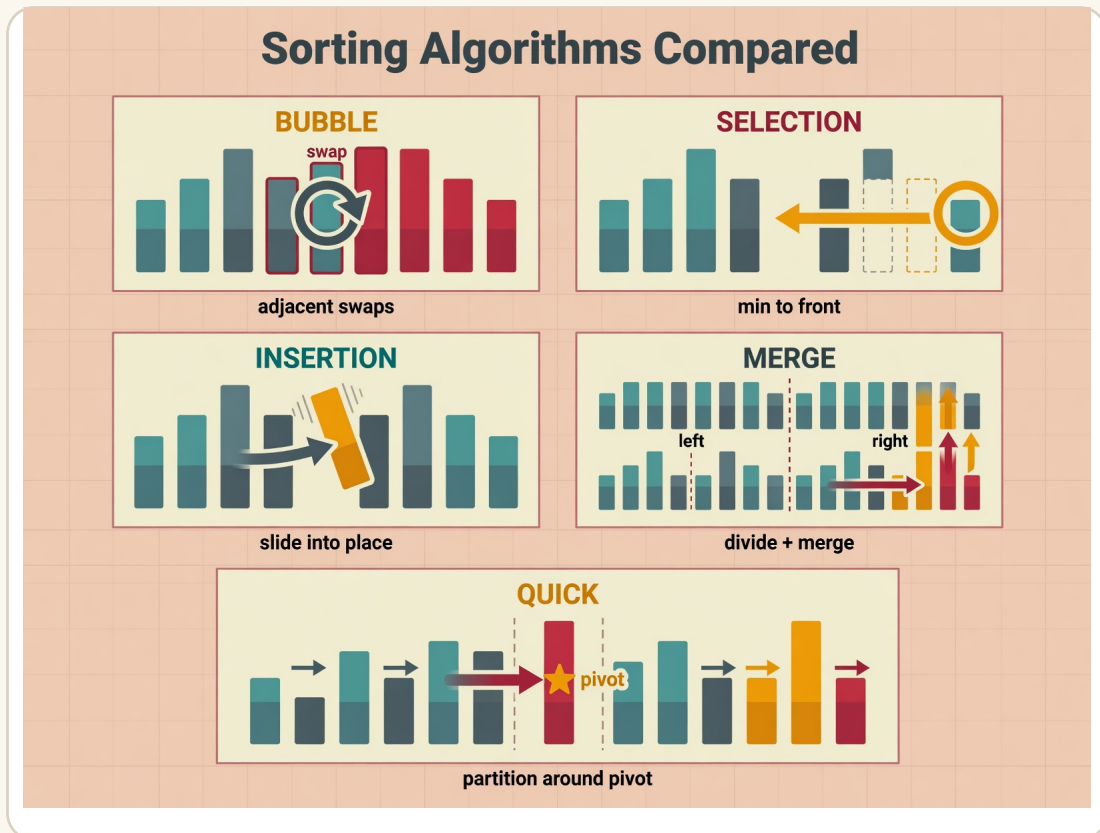
الفصل الرابع — الخوارزميات: الفرز والبحث

الخوارزمية إجراء خطوة بخطوة لحل مشكلة محددة. والفرز هو ترتيب العناصر تصاعديا أو تنازليا لتسهيل البحث والتحليل والعرض.

خوارزميات الفرز الخمس

الذاكرة	التعقيد الزمني	الفكرة	الخوارزمية
$O(1)$	$O(n^2)$ — ■■■■■■ ■■■■■■	قارن الجارين وبدلهما إن لزم	Bubble
$O(1)$	$O(n^2)$ ■■■■■■	ابحث عن الأصغر وضعه في البداية	Selection
$O(1)$	$O(n^2)$ — ■■■■■■ ■■■■■■	أدخل كل عنصر في موضعه الصحيح	Insertion
$O(n)$	$O(n \log n)$ ■■■■■■	اقسم لنصفين، رتبهما، ثم ادمج	Merge
$O(n \log n)$	■■■■■ $O(n \log n)$ ■■■■■■ ■■■■■■	اختر محورا وقسم العناصر حوله	Quick

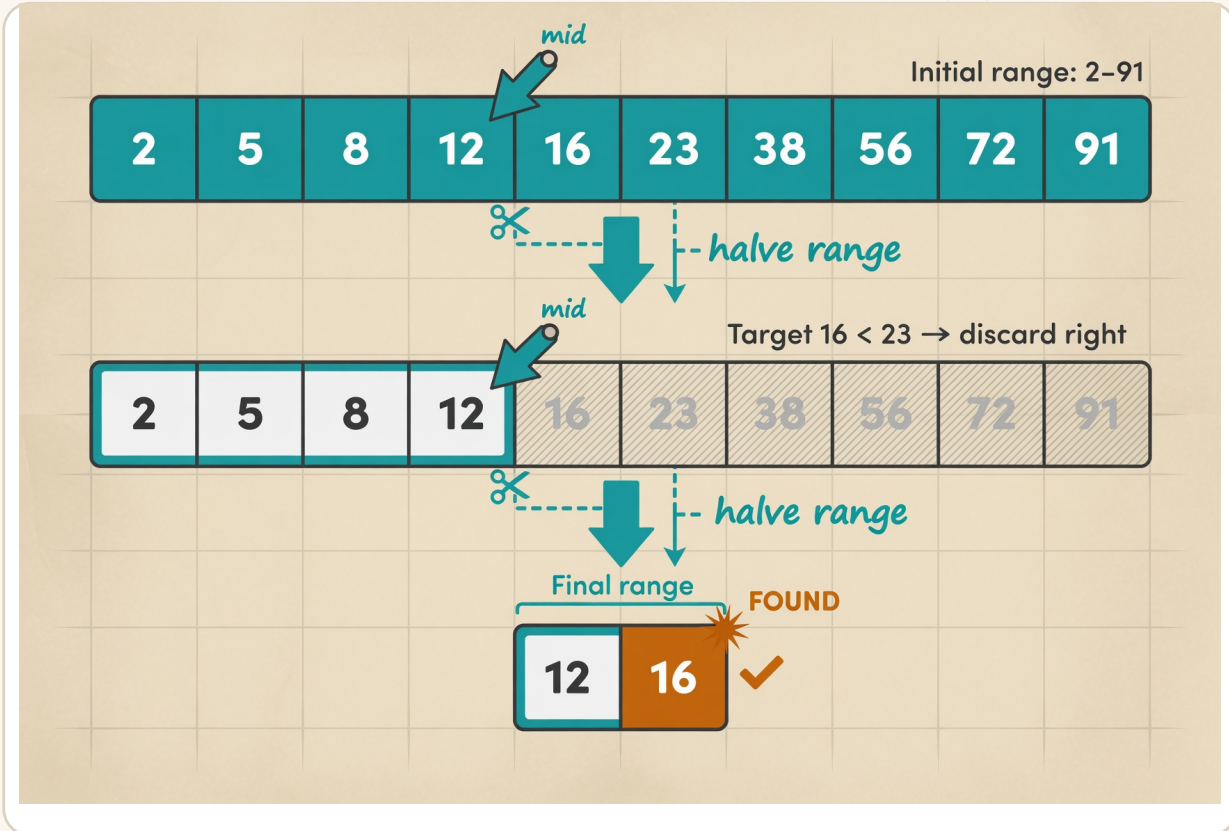
ملاحظة مهمة: خوارزمية Quick Sort هي الأسرع عمليا في أغلب الحالات، لكن اختيار محور سيئ يحولها إلى $O(n^2)$. دائما مقابل ذاكرة إضافية $O(n \log n)$ فتضمن Merge Sort $O(n)$.



الفرق الجوهرى: Bubble و Selection و Insertion تعمل بمقارنات محلية، بينما Merge و Quick تستخدمان استراتيجية فرق تسد.

خوارزميات البحث

- البحث الخطي Linear Search: افحص العناصر واحدا واحدا - تعقيده $O(n)$ ولا يتطلب ترتيبا مسبقا.
- البحث الثنائي Binary Search: يتطلب مصفوفة مرتبة؛ قارن المنتصف بالهدف ثم تخلص عن نصف العناصر في كل خطوة - تعقيده $O(\log n)$.
- قوة التنصيف: البحث الثنائي في مليون عنصر يحتاج نحو 20 مقارنة فقط، لأن كل خطوة تحذف نصف الاحتمالات.



كل خطوة تحذف نصف النطاق - لهذا يبقى المنحنى منخفضا مهما كبرت البيانات.

البحث في الرسوم البيانية (إشارة من المحاضرة)

- البحث بالعرض BFS: يستكشف مستوى مستوى ويستخدم الطابور - التعقيد $O(V+E)$.
- البحث بالعمق DFS: يغوص في فرع حتى نهايته ويستخدم المكس - التعقيد $O(V+E)$.

الفصل الخامس — ورقة الغش قبل الامتحان

- الوصول لعنصر المصفوفة $O(1)$ لأن الذاكرة متصلة، والبحث فيها $O(n)$.
- المكس LIFO والطابور FIFO — لا تخلطهما أبداً.
- العودة بلا حالة أساس = Stack Overflow.
- البحث الثنائي لا يعمل إلا على بيانات مرتبة.
- $O(n^2)$ الأسرع عملياً وأساء حالاته Quick Sort. $O(n \log n)$ ضمان Merge Sort.
- $O(V+E)$ مع المكس — وكلاهما DFS مع الطابور و BFS.

كيف تختار في السؤال؟

إذا سئلت عن أقل ذاكرة إضافية: فكر في Bubble أو Selection أو Insertion لأنها $O(1)$. وإذا سئلت عن ضمان السرعة مهما كانت المدخلات: Merge Sort هي الجواب الآمن.

تم إعداد هذا الملخص بمنهجية الحلقات الاستدلالية (تفكير.md): فهم ثم سياق ثم تحليل عميق ثم صياغة ثم تدقيق — من أربع محاضرات مرفوعة في المستودع (1، 2، 4، 5 و 6).